Design Patterns - Lab 4

Spring Dependency Injection Framework. Controllers

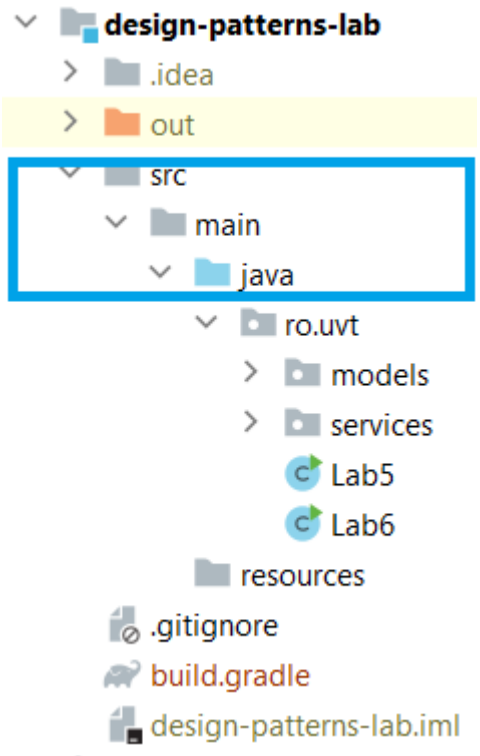
Goal

Explore the dependency injection framework in Spring.

Tasks

If you have created the project using Spring Boot Initializr as described in lab 1 then the strikethrough steps below are not needed.

1. ~~We need to transform our project to a Spring Boot project using Gradle as build manager. There are few steps we need to take before opening the project in IntelliJ IDEA. First thing first is to get a proper project structure. Make sure that your source code is organized correctly, meaning that following a folder structure as in the picture below~~



2. Next, create a build.gradle file in the root of your project folder with the following content.

```
buildscript {
    repositories {
        mavenCentral()
    }
}

plugins {
    id 'org.springframework.boot' version '2.3.3.RELEASE'
    id 'io.spring.dependency-management' version '1.0.8.RELEASE'
    id 'java'
}

repositories {
    mavenCentral()
    jcenter()
}

sourceCompatibility = 1.8

targetCompatibility = 1.8

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'

    compileOnly 'org.projectlombok:lombok:1.18.22'
    annotationProcessor 'org.projectlombok:lombok:1.18.22'

    testCompileOnly 'org.projectlombok:lombok:1.18.22'
    testAnnotationProcessor 'org.projectlombok:lombok:1.18.22'
}
```

3. Discussion of the build.gradle file:
    - Declares the **repositories** for external (library) dependencies
    - **plugins** are the processes ran against your project (e.g., java compiles the Java source code and builds the .jar file)
    - **dependencies** specify project's external dependencies;
        - we made our project a Web-based REST API by including spring-boot-started-web (need it later)

4. Now, it is time to open your gradle project in IntelliJ IDEA. Start IntelliJ IDEA and close any project you have open. From the main dialog, select *Open or Import* and then select the build.gradle file from your main project folder and then select *Open as a project*. Next, select *Import Gradle project* from the toast popup bottom-right corner.

……

.....

5. Create a new subpackage (difexample…) where examples is this lab will go and create our first component there (more about Spring @Component annotation here: https://www.journaldev.com/21429/spring-component); using `@Component` informs the DI container to manage a **unique** instance of our class

```
import org.springframework.stereotype.Component;

@Component
public class SingletonComponent {

   public SingletonComponent() {
       System.out.println(
            "SingletonComponent::SingletonComponent = " + this);
   }

   public void operation() {
       System.out.println(
            "Invoked SingletonComponent::operation() on " + this);
   }
}
```

Q: What is the difference between the unique instance managed by Spring and Singleton pattern?

.....
.....
.....

6. Create a second Component that will have **a new instance** every time an instance is required from DI container (in the same package)

```
import
org.springframework.beans.factory.config.ConfigurableBeanFactory;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class TransientComponent {
```

```
    public TransientComponent() {
          System.out.println("TransientComponent::TransientComponent = "
+ this);
    }

    public void operation() {
          System.out.println("Invoked TransientComponent::operation() on
" + this);
    }
}
```

7. Create now a class that needs an instance of SingletonComponent and an instance of TransientComponent to do its job; we will inject these instances using `@Autowired` annotation and the DI container will take care of providing what we need:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class ClientComponent {
    private final TransientComponent tc;
    private final SingletonComponent sc;

    @Autowired
    public ClientComponent(TransientComponent tc,
                           SingletonComponent sc) {
        this.tc = tc;
        this.sc = sc;
      System.out.println("ClientComponent::ClientComponent = " + this);
        System.out.println("    o SingletonComponent = " + sc);
        System.out.println("    o TransientComponent = " + tc);
    }

    public void operation() {
         System.out.println("Invoked ClientComponent::operation() on " +
this);
        System.out.println("    o SingletonComponent = " + sc);
        System.out.println("    o TransientComponent = " + tc);
    }
}
```

8. Use all these components in the main function and see how DI container manage instances of your classes. MySpringApplication should be created in the top package of your application/project (e.g., ro.uvt); in this way the `@SpringBootApplication` will be able to locate all the `@Component`-annotated classes; otherwise you need to add the folders to be scanned in a separate annotation

```java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import ro.uvt.difexamples.ClientComponent;
import ro.uvt.difexamples.SingletonComponent;
import ro.uvt.difexamples.TransientComponent;

@SpringBootApplication
public class MySpringApplication {

  public static void main(String[] args) {
     //
     // Run the main function and inspect the output in console
     // to learn about the lifecycle of objects within the
     // Spring Dependency Injection Context
     //

     // Gets a handle of dependency injection context
     ApplicationContext context =
            SpringApplication.run(MySpringApplication.class, args);

     // Gets an instance of TransientComponent from the DI context
     TransientComponent transientBean =
            context.getBean(TransientComponent.class);
     transientBean.operation();

     // Note that every time an instance is required,
     // the DI context creates a new one
     transientBean = context.getBean(TransientComponent.class);
     transientBean.operation();

     // Gets an instance of SingletonComponent from the DI context
     // Note that the unique instance was created while
     // application was loaded, before creating
     // the transient instances
     SingletonComponent singletonBean =
            context.getBean(SingletonComponent.class);
     singletonBean.operation();

     // Note that every time an instance is required,
     // the DI returns the same unique one
     singletonBean = context.getBean(SingletonComponent.class);
     singletonBean.operation();

     // Gets an instance of another class that
     // requires singleton/transient components
     // Note where this instance was created and what beans
     // were used to initialize it
     ClientComponent c = context.getBean(ClientComponent.class);
     c.operation();
```

```
        // One can also request an instance from DI context by name
        c = (ClientComponent)context.getBean("clientComponent");
        c.operation();
    }
}
```

9. Let's add our first controller, `HelloController` (create it under `controllers` package) and return "Hello from Spring Boot" message for the root endpoint accessible at http://localhost:8080.

10. Inject the `ClientComponent` in this controller and return "Hello from ClientComponent = ClientComponent@8d7b252"

Tip: declare the controller as a `record` (immutable object), or use Lombok's `@RequiredArgsConstructor(onConstructor_ = @Autowired)` to avoid boilerplate code in constructor.

11. Try to inject different instances of ClientComponent in the HelloController class. Explain the results

……
……
……